Mongoose

Mongoose

A library that creates a connection between MongoDB & Node.js JavaScript Runtime Environment

It is an ODM (Object Data Modeling) Library.

Installation

npm i mongoose

Schema

Schema defines the shape of the documents within that collection.

```
const userSchema = new mongoose.Schema({
  name: String,
  email: String,
  age: Number,
});
```

Models

Model in mongoose is a class with which we construct documents.

```
const User = mongoose.model("User", userSchema);
```

Model

collection

INSERT

Inserting One

```
const user1 = new User({ name: "Adam", email: "adam@yahoo.in", age: 48 });
const user2 = new User({ name: "Eve", email: "eve@google.com", age: 48 });
```

user1.save() *//to save in DB*

user2.save() *//to save in DB*

```
const userSchema = new mongoose.Schema({
    name:String,
    email:String,
    age:Number,
});
const User = mongoose.model("User",userSchema);
const Employee = mongoose.model("Employee",userSchema);
const user1 = new User({
    name:"Annu",
    email:"annumathur003@gmail.com",
    age:21,
});
const user2 = new User({
    name:"Ana",
    email:"annumathur004@gmail.com",
    age:22,
});
user2.save().then((res)=>{
    console.log(res);
}).catch((err)=>{
    console.log(err);
}); //this is also asynchornous function and return promise
```

```
test> db.users.find()
[[
  {
    _id: ObjectId('69e0886ac6e2a50a31deb3f7'),
    name: 'Annu',
    email: 'annumathur003@gmail.com',
    age: 21,
    __v: 0
  }
]
test> db.users.find()
[[
  {
    _id: ObjectId('69e0886ac6e2a50a31deb3f7'),
    name: 'Annu',
    email: 'annumathur003@gmail.com',
    age: 21,
    __v: 0
  },
  {
    _id: ObjectId('69e089027ed8bfd3ee903a6e'),
    name: 'Ana',
    email: 'annumathur004@gmail.com',
    age: 22,
    __v: 0
  }
]
test>
```

INSERT

Inserting Multiple

```
User.insertMany([
  { name: "Tony", email: "tony@gmail.com", age: 50 },
  { name: "Bruce", email: "bruce@gmail.com", age: 47 },
  { name: "Peter", email: "peter@gmail.com", age: 30 },
]).then((data) => {
  console.log(data);
});
```

annumathur003@gmail.com

```
User.insertMany([
    {name:"Anu",email:"anu23@gmail.com",age:20},
    {name:"Anshi",email:"anshi123@gmail.com",age:22},
    {name:"Anamika",email:"anamika443@gmail.com",age:23},

]).then((res)=>{
    console.log(res);
});
```

```
test> db.users.find()
[
  {
    _id: ObjectId('69e0886ac6e2a50a31deb3f7'),
    name: 'Annu',
    email: 'annumathur003@gmail.com',
    age: 21,
    __v: 0
  },
  {
    _id: ObjectId('69e089027ed8bfd3ee903a6e'),
    name: 'Ana',
    email: 'annumathur004@gmail.com',
    age: 22,
    __v: 0
  },
  {
    _id: ObjectId('69e08a637e89a54c78af0ec2'),
    name: 'Anu',
    email: 'anu23@gmail.com',
    age: 20,
    __v: 0
  },
  {
    _id: ObjectId('69e08a637e89a54c78af0ec3'),
    name: 'Anshi',
    email: 'anshi123@gmail.com',
    age: 22,
    __v: 0
  },
  {
    _id: ObjectId('69e08a637e89a54c78af0ec4'),
    name: 'Anamika',
    email: 'anamika443@gmail.com',
    age: 23,
    __v: 0
  }
]
test>
```

Note

Mongoose uses Operation Buffering

Mongoose lets you start using your models immediately, without waiting for mongoose to establish a connection to MongoDB.

FIND

Model.find() //returns a Query Object (thennable)

***Mongoose Queries are not promises. But they have a .then( )**

```
User.find().then((data) => {
  console.log(data);
});
```

```
User.find({ age: { $gte: 47 } }).then((data) => {
  console.log(data);
});
```

FIND

Model.findOne() //returns a single result

```
User.findOne({ age: { $gte: 47 } }).then((data) => {
  console.log(data);
});
```

Model.findById() //commonly used

`Model.findById()`

Parameters:

- `id` «Any» value of `_id` to query by
- `[projection]` «Object|String|Array[String]» optional fields to return, see `Query.prototype.select()`
- `[options]` «Object» optional see `Query.prototype.setOptions()`

Returns:

- «Query»

UPDATE

Model.updateOne()

```
User.updateOne({ name: "Bruce" }, { age: 49 }).then((res) => {
  console.log(res);
});
```

Model.updateMany()

```
User.updateMany({ age: { $gt: 45 } }, { age: 45 }).then((res) => {
  console.log(res);
});
```

UPDATE

Model.findOneAndUpdate()

```
User.updateMany({ age: { $gt: 45 } }, { age: 45 }).then((res) => {
  console.log(res);
});
```

```
User.findOneAndUpdate({ name: "Tony" }, { age: 60 }, { new: true }).then(
  (data) => {
    console.log(data);
  }
);
```

Model.findByIdAndUpdate()

DELETE

Model.deleteOne() //returns count

```
User.deleteOne({ name: "Adam" }).then((res) => {
  console.log(res);
});
```

Model.deleteMany() //returns count

```
User.deleteMany({ age: { $gt: 40 } }).then((res) => {
  console.log(res);
});
```

```
User.deleteOne({name:"Anamika"}).then((res)=>{
    console.log(res);
});
```

```
gauravmathur@Annus-Mac MONGO % node index.js
connection sucessful
{ acknowledged: true, deletedCount: 1 }
```

DELETE

Try Yourself

Model.findByIdAndDelete()

Model.findOneAndDelete()

annumathur003@gmail.com

Schema Validations

Basically, Rules for Schema

```
const bookSchema = mongoose.Schema({
  title: {
    type: String,
    required: true,
  },
  author: {
    type: String,
  },
  price: {
    type: Number,
  },
});
```

Schema Validations

SchemaType Options

```
const bookSchema = mongoose.Schema({
  title: {
    type: String,
    required: true,
  },
  author: {
    type: String,
  },
  price: {
    type: Number,
    min: [1, "please enter a valid price"],
  },
  discount: {
    type: Number,
    default: 0,
  },
  genre: [String],
  category: {
    type: String,
    enum: ["fiction", "non-fiction"],
```